

TA-IND-02 — Informe Técnico Individual

Análisis de Consistencia y Replicación en BDD Distribuidas

Estudiante: Keyla Betzabe Bedón Viteri

Equipo PFC: BCEL – SGA Escuela de Educación Básica Provincias Unidas

Microservicio a cargo del autor: Docente

Docente: Ing. Guerrero Ulloa Gleiston Cicero, PhD

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad 3 – Bases de Datos Distribuidas

Período académico: 2026–2027

14 de julio de 2026

Resumen

Este informe analiza el modelo de consistencia y replicación adoptado por el microservicio Docente dentro del Proyecto de Fin de Curso (PFC) del equipo BCEL, un Sistema de Gestión Académica (SGA) para la Escuela de Educación Básica Provincias Unidas, organizado en microservicios independientes (Principal/Autenticación, Secretaría, Docente, Soporte técnico). A diferencia de un esquema de base de datos compartida, el equipo BCEL adoptó el patrón *database-per-service*: el microservicio Principal opera sobre su propia instancia PostgreSQL (Spring Boot) y el microservicio Docente opera sobre una instancia PostgreSQL independiente en Supabase (Django REST Framework), comunicándose mediante REST y gRPC, con réplicas de lectura configuradas sobre la base del microservicio Docente. Este informe caracteriza dicho modelo a la luz del espectro de consistencia y de los marcos CAP y PACELC, argumentando que la ausencia de claves foráneas físicas entre servicios traslada la integridad referencial a llamadas RPC síncronas contra el microservicio Principal, mientras que la replicación asíncrona de lectura introduce un riesgo concreto de *staleness* en los reportes académicos (boletas, promedios) que el microservicio genera. Se evalúan dos alternativas replicación síncrona y desacoplamiento mediante caché local con eventos frente al modelo vigente, concluyendo que este último es razonable para la fase académica del PFC pero requiere mitigaciones concretas antes de un despliegue con tráfico real.

1 Introducción

El PFC del equipo BCEL implementa el Sistema de Gestión Académica (SGA) de la Escuela de Educación Básica Provincias Unidas, organizado en microservicios independientes que se comunican mediante un microservicio Principal que actúa como gateway de autenticación y de comunicación inter-servicio. El autor de este informe es responsable del microservicio **Docente**, construido sobre Django y Django REST Framework, cuya base de datos PostgreSQL reside en una instancia gestionada de Supabase bajo el esquema `sga_docente`.

El microservicio Docente administra nueve módulos de negocio: Actividades, Calificaciones, Asistencias, Resumen de asistencia, Promedios trimestrales, Promedios anuales, Detalle de promedios anuales, Seguimiento académico y Períodos de evaluación. Estos módulos concentran la lógica de evaluación del estudiante incluida la fórmula de promedio trimestral ($\text{promedio_trimestral} = \text{promedio_formativo} \times 0.70 + \text{nota_sumativa} \times 0.30$), el cálculo del promedio anual y la conversión de notas cuantitativas a la escala cualitativa institucional (DAR, AAR, PAR, NAR) por lo que la corrección e integridad de sus datos tiene impacto directo sobre documentos oficiales del estudiante, como la boleta trimestral y el concentrado de calificaciones.

A diferencia del caso analizado en otros equipos del curso, en los que varios microservicios comparten una única instancia de base de datos separada únicamente por esquemas, el equipo BCEL decidió que cada microservicio posea su propia instancia física de base de datos: el microservicio Principal opera sobre PostgreSQL propio (Spring Boot) y el microservicio Docente opera sobre una instancia PostgreSQL independiente (Django, Supabase), comunicándose mediante REST y gRPC. Además, sobre la base de datos del microservicio Docente se han configurado réplicas de lectura. Esta combinación *database-per-service* más replicación de lectura ocupa una posición particular dentro del espectro de consistencia que este informe caracteriza con precisión.

El presente informe responde a la pregunta orientadora de la unidad: **¿es el modelo de consistencia elegido el óptimo para el dominio del sistema?** El objetivo es caracterizar formalmente el modelo *database-per-service* con réplicas de lectura implementado para el microservicio Docente, contrastarlo con al menos dos alternativas documentadas en la literatura, y sostener una postura razonada sobre su optimalidad, sus riesgos y las mitigaciones aplicables.

2 Marco Teórico

2.1 El espectro de modelos de consistencia

La consistencia en sistemas de almacenamiento distribuido no es una propiedad binaria, sino un espectro ordenado por la fuerza de las garantías que ofrece a los clientes de las réplicas [1]. En un extremo se ubica la linealizabilidad (o consistencia fuerte), que exige que toda operación parezca ejecutarse de forma instantánea en un punto entre su invocación y su reconocimiento, de modo que todas las réplicas perciban un único orden total compatible con el tiempo real [1]. Le sigue la consistencia secuencial, que exige un orden total acordado por todos los clientes pero relaja la restricción de tiempo real, y la consistencia causal, que solo exige preservar el orden entre operaciones causalmente relacionadas. Particularmente relevante para el caso del microservicio Docente es la garantía de *lectura de los propios cambios* (read-your-writes), una de las garantías centradas en el cliente que Viotti y Vukolić distinguen de las garantías centradas en los datos: un docente que acaba de registrar una calificación espera que, al generar inmediatamente después un reporte, esa calificación ya esté reflejada [1]. En el extremo más débil se encuentra la consistencia ventual, que únicamente garantiza que, en ausencia de nuevas escrituras, todas las réplicas convergerán al mismo valor. Bailis y Ghodsi remarcan que, en la práctica, la consistencia eventual y sus variantes son adoptadas por compromisos concretos de latencia y disponibilidad, y que la elección entre un modelo más fuerte o más débil debe hacerse considerando explícitamente esos compromisos, tal como los describen CAP y PACELC [2].

2.2 Teoremas de compromiso: CAP y PACELC

El teorema CAP, formalizado por Gilbert y Lynch a partir de la conjetura original de Brewer, establece que ante una partición de red un sistema distribuido no puede garantizar simultáneamente consistencia y disponibilidad; debe sacrificar una de las dos mientras la partición persiste [3]. Abadi observó que esta lectura es incompleta, dado que CAP solo describe el comportamiento del sistema durante una partición, un evento poco frecuente en la mayoría de los despliegues; propuso entonces la formulación PACELC: en presencia de una Partición (P), el sistema elige entre Availability (A) y Consistencia (C); pero *Else* (E), en operación normal sin particiones, el sistema elige entre Latencia (L) y Consistencia (C) [4]. Una revisión reciente sobre el diseño integral de sistemas de bases de datos distribuidas remarca que este compromiso debe leerse de forma continua a lo largo de todo el ciclo de vida del sistema, y no como una decisión puntual de arquitectura [5].

2.3 Estrategias de replicación

La estrategia de replicación determina qué nivel de consistencia es alcanzable en la práctica. En la replicación primario-respaldo (*leader-follower*), un único nodo líder acepta todas las escrituras y las propaga a uno o más nodos seguidores; si la propagación es síncrona, el sistema puede sostener consistencia fuerte a costa de mayor latencia de escritura, mientras que si es asíncrona se introduce un retraso de replicación (*staleness* acotado) a cambio de menor latencia de escritura y mayor disponibilidad de lectura [2]. Los sistemas de referencia industrial ilustran los extremos de este espectro: Dynamo, el almacén clave-valor de Amazon, prioriza explícitamente la disponibilidad mediante replicación sin líder por quorum y consistencia eventual con resolución de conflictos del lado del cliente [6]; en el extremo opuesto, Spanner ofrece consistencia externa apoyándose en TrueTime, un servicio de sincronización de relojes con incertidumbre acotada [7].

2.4 El patrón *database-per-service* y la integridad referencial entre microservicios

Laigner, Zhou y Vaz Salles argumentan que el patrón *database-per-service*, si bien favorece el aislamiento de fallos a nivel de microservicio, traslada una carga sustancial de gestión de datos hacia la capa de aplicación, obligando a los equipos de desarrollo a resolver manualmente problemas de integridad referencial y consistencia entre servicios que antes resolvía el motor de base de datos de forma nativa [8]. Kanwal et al. muestran que, al eliminarse las restricciones de clave foránea nativas entre los límites de los microservicios, la integridad referencial debe sostenerse mediante mecanismos alternativos validación por API síncrona, comunicación orientada a eventos, o el patrón Saga con transacciones compensatorias cada uno con distintos compromisos entre consistencia, disponibilidad y complejidad operativa [9]. Shabani et al. complementan esta discusión desde una perspectiva arquitectónica más general, mostrando que la descomposición de un sistema monolítico en microservicios introduce ganancias de escalabilidad y mantenibilidad, pero a costa de mayor complejidad de coordinación entre los componentes distribuidos [10].

3 Análisis del Modelo de Consistencia del PFC

3.1 Caracterización del modelo implementado

El equipo BCEL adoptó, para el microservicio Docente, el patrón *database-per-service* en su forma más estricta: el microservicio Principal (Spring Boot) opera sobre su propia instancia PostgreSQL, y el microservicio Docente (Django REST Framework) opera sobre una instancia PostgreSQL físicamente independiente, alojada en Supabase bajo el esquema *sga_docente*. No existe una base de datos compartida entre servicios ni claves foráneas físicas que crucen de una instancia a otra. Sobre la instancia del microservicio Docente se ha configurado, además, al menos una réplica de lectura.

La Figura 1 ilustra la topología: dos instancias PostgreSQL físicamente independientes una por microservicio, comunicadas mediante REST/gRPC, con una réplica de lectura desprendida de la instancia primaria del microservicio Docente.

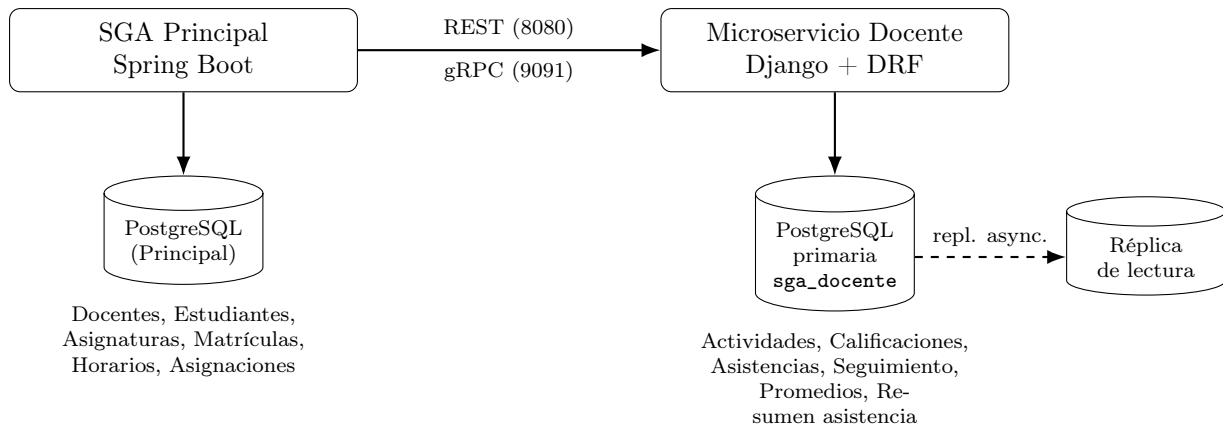


Figura 1: Topología del modelo de datos del microservicio Docente dentro del PFC de BCEL: dos instancias PostgreSQL físicamente independientes (patrón *database-per-service*), comunicadas por REST/gRPC, con una réplica de lectura sobre la instancia primaria del microservicio Docente. No existen claves foráneas físicas entre las dos bases de datos.

Esta separación de instancias tiene un correlato exacto en la propiedad de los datos. El microservicio Principal administra las entidades de *catálogo* y de *matrícula* del sistema: Docentes, Estudiantes, Asignaturas, Matrículas, Horarios y Asignaciones, mientras que el microservicio Docente administra únicamente las entidades de *evaluación* generadas durante el período lectivo: Actividades, Calificaciones, Asistencias, Seguimiento académico, Promedios y Resumen de asistencia.

En consecuencia, todo identificador que el microservicio Docente recibe y persiste como referencia a un estudiante, una asignatura o una asignación docente-materia (`id_matricula`, `id_asignacion`) apunta a un registro cuya fuente de verdad reside físicamente en la base de datos del microservicio Principal, y no en el esquema `sga_docente`.

Esta correspondencia exacta entre módulos y microservicios es la que hace concreto, y no solo teórico, el problema de integridad referencial entre servicios descrito por Kanwal et al. y Laigner et al. para el patrón *database-per-service*: no existe ni puede existir, al tratarse de instancias físicamente separadas una clave foránea física de `calificaciones.id_matricula` hacia `matriculas.id` en el sentido en que la definiría un motor relacional dentro de una misma base de datos.

La comunicación entre servicios sigue dos caminos complementarios. Primero, el frontend del microservicio Docente nunca invoca gRPC directamente: toda solicitud pasa por el microservicio Principal, que actúa como *gateway* (REST en el puerto 8080 hacia el frontend, gRPC en el puerto 9091 hacia Docente para las consultas que el Principal necesita resolver). Segundo, y en sentido inverso, el propio microservicio Docente valida el contexto y los permisos del docente autenticado llamando de vuelta al microservicio Principal mediante un servicio gRPC dedicado (`TeacherContextService`, expuesto en el puerto 9092). Esta segunda ruta es la que resuelve, en tiempo de escritura, la integridad referencial que en un esquema compartido resolvería una clave foránea: cuando el microservicio Docente registra una calificación o una asistencia asociada a un `id_matricula` o a un `id_asignacion`, esos identificadores no existen físicamente en el esquema `sga_docente`; son referencias lógicas a entidades cuya fuente de verdad reside en otro microservicio, y su validez se confirma o se asume válida sin una llamada explícita adicional en el momento de la solicitud HTTP/gRPC, no mediante una restricción del motor de base de datos.

El Listado 1 muestra uno de los *endpoints* de negocio del microservicio Docente cuyo cálculo depende enteramente de datos que residen en su propia instancia, pero cuya validez de entrada (identidad del estudiante y de la asignación) depende de una fuente externa.

Listing 1: Endpoint de cálculo de promedio trimestral del microservicio Docente.

```
POST /api/docente/promedios-trimestrales/calcular/
Content-Type: application/json

{
  "id_matricula": 1,
  "id_asignacion": 10,
  "id_periodo": 1,
  "nivel": "EGB"
}

# promedio_trimestral = promedio_formativo * 0.70 + nota_sumativa * 0.30
```

3.2 Justificación técnica ligada al dominio bajo CAP/PACELC

Bajo el teorema CAP, cada microservicio, al operar sobre su propia instancia, puede tolerar de forma independiente una partición entre las dos bases de datos: si el microservicio Principal queda inalcanzable, el microservicio Docente puede seguir aceptando escrituras sobre su propia base (Actividades, Calificaciones, Asistencias), aunque las operaciones que dependen de `TeacherContextService` para validar el contexto del docente quedarían degradadas o bloqueadas [3]. Esta es la ventaja característica del patrón *database-per-service* frente al esquema compartido: el aislamiento de fallos es real, porque no existe un único nodo de base de datos cuya caída afecte a ambos microservicios simultáneamente.

Bajo PACELC, el compromiso relevante no ocurre entre los dos microservicios – que no comparten motor de base de datos – sino *dentro* de la instancia del microservicio Docente, entre su nodo primario y la réplica de lectura configurada. Si esa replicación es asíncrona (el caso típico en despliegues gestionados como Supabase), el sistema se ubica en la categoría PA/EL: ante una partición, prioriza disponibilidad (las lecturas pueden seguir sirviéndose desde la réplica aunque el primario esté degradado); en operación normal, prioriza latencia sobre consistencia, porque las lecturas desde la réplica son más rápidas pero pueden reflejar un estado ligeramente anterior al del primario [4]. Esta caracterización como asíncrona es, en este momento, una asunción de este informe que el equipo debe confirmar explícitamente, ya que Supabase permite en principio configurar replicación tanto síncrona como asíncrona según el nivel de servicio contratado.

Esta caracterización no es una discusión abstracta para el microservicio Docente: los módulos de *Reportes* (boleta trimestral, RAE, concentrado) y de *Promedios anuales* agregan datos que pudieron haberse escrito apenas segundos antes – por ejemplo, un docente que corrige una calificación y de inmediato solicita el concentrado del curso. Si esas lecturas de reporte se sirven desde la réplica y la replicación es asíncrona, el reporte generado podría no reflejar la corrección más reciente, violando precisamente la garantía de *lectura de los propios cambios* descrita en la Sección 2.1 [1]. Los *endpoints* de cálculo (`/promedios-trimestrales/calcular/`, `/promedios-anuales/calcular/`, `/resumen-asistencia/calcular/`), en cambio, al ejecutarse dentro de la misma solicitud HTTP que probablemente resuelve Django contra el primario, no deberían verse afectados por este riesgo; el riesgo se concentra en los listados y reportes de solo lectura que sí podrían enrutarse hacia la réplica.

4 Evaluación de Alternativas

Se contrastan dos alternativas frente al modelo vigente (*database-per-service* con réplica de lectura asíncrona y validación cruzada por gRPC síncrono), seleccionadas por atacar cada una de las dos fuentes de riesgo identificadas en la Sección 3.2: el riesgo de *staleness* en reportes y el acoplamiento síncrono entre Docente y Principal para la integridad referencial.

4.1 Alternativa A: replicación síncrona para los reportes académicos

En este esquema, la réplica de lectura del microservicio Docente pasaría de asíncrona a síncrona (*synchronous streaming replication*), de modo que toda escritura confirmada en el primario quede garantizada también en la réplica antes de reconocerse al cliente. Esto elimina el riesgo de *staleness* descrito para los módulos de Reportes y Promedios anuales, a costa de un incremento en la latencia de cada escritura (calificación, asistencia) proporcional a la latencia de red hacia la réplica, y de una reducción de disponibilidad de escritura si la réplica queda inalcanzable [2].

4.2 Alternativa B: caché local de contexto académico con actualización por eventos

En este esquema, en lugar de que el microservicio Docente valide el `id_matricula` y el `id_asignacion` mediante una llamada gRPC síncrona a `TeacherContextService` en cada escritura, mantendría una copia local (caché) de los datos mínimos de matrícula y asignación necesarios para sus propias validaciones, actualizada de forma asíncrona mediante eventos publicados por el microservicio Principal cada vez que cambian esos datos. Esto reduce el acoplamiento síncrono entre ambos microservicios Docente podría seguir operando aunque Principal esté temporalmente caído pero introduce consistencia eventual entre el estado real de la matrícula/asignación y la copia local que usa Docente para validar sus propias escrituras, un riesgo de integridad referencial similar al que documentan Kanwal et al. para arquitecturas Saga [9].

4.3 Tabla comparativa

La Tabla 1 resume el contraste entre el modelo vigente y las dos alternativas según cinco criterios técnicos.

Tabla 1: Comparación del modelo vigente (database-per-service + réplica asíncrona) frente a dos alternativas.

Criterio	Vigente: DB por servicio + réplica async.	A: réplica síncrona	B: caché local + eventos
Consistencia	Fuerte en primario; <i>staleness</i> posible en reportes vía réplica	Fuerte en primario y réplica (sin <i>staleness</i>)	Eventual entre Principal y la copia local de Docente
Disponibilidad	Alta; aislamiento real de fallos entre microservicios	Escritura degradada si la réplica falla	Muy alta; Docente opera aunque Principal esté caído
Latencia	Baja en escritura; baja en lectura de réplica	Mayor en cada escritura (confirmación en réplica)	Baja en escritura y en validación (sin RPC síncrono)
Integridad referencial	Vía RPC síncrono a Principal (acoplamiento fuerte)	Igual que el modelo vigente (no cambia)	Requiere eventos y tolerar copias desactualizadas
Complejidad operativa	Moderada (dos instancias, un tipo de réplica)	Moderada-alta (configuración de réplica síncrona)	Alta (bus de eventos, esquemas de mensajes, reconciliación)

5 Discusión Crítica

La respuesta a la pregunta orientadora es que el modelo vigente para el microservicio Docente *database-per-service* con réplica de lectura y validación cruzada por gRPC síncrono es razonable para la fase académica del PFC, aunque no óptimo a mediano plazo. Es razonable porque preserva la ventaja central del patrón *database-per-service*: el aislamiento de fallos entre el microservicio Docente y el resto del sistema es real y no depende de la disponibilidad de un motor de base de datos compartido, a diferencia de lo que ocurriría en un esquema de instancia única [8]. No es óptimo a mediano plazo por dos razones concretas y verificables en el propio dominio del microservicio Docente. Primera, bajo PACELC, la réplica de lectura si es asíncrona, como se asume en este informe introduce un riesgo real de *staleness* exactamente en los módulos donde la exactitud del dato es más crítica: los reportes académicos oficiales (boleta trimestral, concentrado) y los promedios anuales, que son documentos que el estudiante y la institución consultan como fuente de verdad [4]. Segunda, bajo el análisis de integridad referencial entre microservicios de Kanwal et al. [9] y Laigner et al. [8], la dependencia del microservicio Docente de una llamada gRPC síncrona a `TeacherContextService` para validar cada escritura reintroduce, por otra vía, parte del acoplamiento que el patrón *database-per-service* buscaba evitar: si el microservicio Principal está lento o inalcanzable, el microservicio Docente podría no poder registrar una calificación aunque su propia base de datos esté perfectamente disponible.

De las dos alternativas evaluadas, ninguna domina a la otra en todos los criterios, por lo que la recomendación depende de cuál de los dos riesgos el equipo BCEL considere más urgente de mitigar antes de un despliegue con tráfico real. Si la prioridad es la exactitud de los reportes académicos oficiales que es, en criterio del autor, el riesgo más directamente ligado al dominio de negocio, dado que un boletín con notas desactualizadas tiene consecuencias académicas y administrativas concretas, la Alternativa A (réplica síncrona) es la mitigación más simple y de menor complejidad operativa. Si en cambio la prioridad es reducir el acoplamiento operativo entre microservicios para escalar el equipo de desarrollo de forma independiente, la Alternativa B es más coherente con la filosofía original de *database-per-service*, aunque exige una inversión de ingeniería (bus de eventos, manejo de reconciliación) que no se justifica mientras el sistema esté en fase académica con tráfico reducido y con un único equipo de desarrollo a cargo de ambos microservicios.

Esta conclusión tiene tres limitaciones que deben reconocerse. Primera, y la más importante, todo el análisis de la Sección 3.2 asume que la réplica de lectura configurada sobre la base del microservicio Docente opera en modo asíncrono; esta es la configuración más habitual en proveedores gestionados como Supabase, pero debe confirmarse explícitamente con la configuración real del proyecto antes de dar por válido el riesgo de *staleness* descrito. Segunda, este informe no pudo determinar, a partir de la documentación disponible del microservicio, si las rutas de lectura de los módulos de Reportes y Promedios anuales están efectivamente enrutadas hacia la réplica o si Django resuelve todas las consultas contra el primario; de ser esto último, el riesgo de *staleness* sería solo teórico y no aplicaría en la implementación actual. Tercera, tampoco se pudo determinar si existe algún mecanismo de reintento o de tolerancia a fallos en la llamada gRPC síncrona hacia `TeacherContextService`; su ausencia agravaría el riesgo de acoplamiento descrito en esta sección. Se recomienda al equipo BCEL documentar explícitamente estas tres decisiones como parte del diseño del microservicio Docente.

6 Conclusiones

1. El microservicio Docente del PFC de BCEL adopta el patrón *database-per-service* en su forma estricta: cada microservicio del SGA opera sobre su propia instancia física de PostgreSQL, sin claves foráneas físicas que crucen entre servicios, complementado con una réplica de lectura sobre la instancia del microservicio Docente.

2. Este modelo preserva una ventaja real de aislamiento de fallos entre microservicios, que un esquema de instancia compartida no ofrecería, pero traslada la integridad referencial entre servicios (validación de matrícula y asignación) a una llamada gRPC síncrona contra el microservicio Principal, reintroduciendo un acoplamiento operativo que debe gestionarse explícitamente.
3. Bajo CAP y PACELC, el principal riesgo del modelo no radica en la partición entre los dos microservicios que operan de forma aislada por diseño, sino en la posible replicación asíncrona dentro de la propia instancia del microservicio Docente, con impacto directo sobre la exactitud de los reportes académicos oficiales.
4. Entre las alternativas analizadas, adoptar replicación síncrona para los módulos de reportes resulta la mitigación más simple frente al riesgo de *staleness*, mientras que desacoplar la validación cruzada mediante una caché local actualizada por eventos ataca el riesgo de acoplamiento operativo, a costa de mayor complejidad de ingeniería.
5. En conjunto, el modelo implementado es adecuado para la fase académica del PFC, siempre que el equipo BCEL confirme el modo de replicación configurado, verifique el enrutamiento de lecturas de los módulos de Reportes y Promedios anuales, y documente explícitamente el manejo de fallos de la validación cruzada por gRPC antes de cualquier despliegue con tráfico real.

7 Declaración de Uso de IA Generativa

Para la elaboración de este informe se utilizó el asistente de IA generativa Claude (Anthropic). Toda la caracterización del modelo de datos realmente implementado en el microservicio Docente, la evaluación de alternativas y la postura crítica frente a la pregunta orientadora fueron elaboradas a partir del conocimiento directo del autor sobre su microservicio y de la documentación técnica del repositorio del proyecto. La IA se utilizó como apoyo de redacción: para dar coherencia al texto, conectar el marco teórico con el análisis propio del dominio, y mejorar la claridad de la redacción final del marco teórico y de la discusión crítica. La postura crítica final del informe (la evaluación de si el modelo es óptimo y la elección entre alternativas) fue revisada y validada por el autor antes de la entrega.

Referencias

- [1] P. Viotti y M. Vukolić, «Consistency in Non-Transactional Distributed Storage Systems,» *ACM Computing Surveys*, vol. 49, n.º 1, art. 19, págs. 1–34, 2016. doi: 10.1145/2926965.
- [2] P. Bailis y A. Ghodsi, «Eventual Consistency Today: Limitations, Extensions, and Beyond,» *Communications of the ACM*, vol. 56, n.º 5, págs. 55–63, 2013. doi: 10.1145/2447976.2447992.
- [3] S. Gilbert y N. Lynch, «Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51–59, 2002. doi: 10.1145/564585.564601.
- [4] D. J. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37–42, 2012. doi: 10.1109/MC.2012.33.
- [5] G. Carvalho, J. Bernardino, B. Cabral y V. Pereira, «A Survey of Holistic Approaches for Distributed Database Systems: From Conceptual Model to Deployment,» *IEEE Access*, 2025, Early access. doi: 10.1109/ACCESS.2025.3587670.

- [6] G. DeCandia *et al.*, «Dynamo: Amazon’s Highly Available Key-value Store,» en *Proc. 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP ’07)*, 2007, págs. 205–220. doi: 10.1145/1294261.1294281.
- [7] J. C. Corbett *et al.*, «Spanner: Google’s Globally Distributed Database,» *ACM Transactions on Computer Systems*, vol. 31, n.º 3, art. 8, págs. 1–22, 2013. doi: 10.1145/2491245.
- [8] R. Laigner, Y. Zhou y M. A. Vaz Salles, «A Distributed Database System for Event-based Microservices,» en *Proc. 15th ACM Int. Conf. on Distributed and Event-Based Systems (DEBS ’21)*, 2021, págs. 25–30. doi: 10.1145/3465480.3466919.
- [9] S. Kanwal, N. R. Chaudhry, R. Choudhary, Y. A. Shaik, P. Yadav y A. Rashid, «Foreign Key Constraints to Maintain Referential Integrity in Distributed Database in Microservices Architecture,» *International Journal of Advanced Computer Science and Applications*, vol. 16, n.º 6, art. 96, 2025. doi: 10.14569/IJACSA.2025.0160696.
- [10] I. Shabani, E. Mëziu, B. Berisha y T. Biba, «Design of Modern Distributed Systems based on Microservices Architecture,» *International Journal of Advanced Computer Science and Applications*, vol. 12, n.º 2, págs. 153–159, 2021. doi: 10.14569/IJACSA.2021.0120220.